Advanced SQL Exercises — Answer Key

Online Retail Store: Ranking, Aggregation, CTEs, MERGE, PIVOT/UNPIVOT

Assumed Database Schema

The exercises reference a standard online retail schema. The tables and key columns assumed throughout this answer key are summarized below.

Table	Key Columns
Customers	CustomerID (PK), Name, Region
Products	ProductID (PK), ProductName, Category, Price
Orders	OrderID (PK), CustomerID (FK), OrderDate
OrderDetails	OrderDetailID (PK), OrderID (FK), ProductID (FK), Quantity
StagingProducts	ProductID, ProductName, Category, Price (used in Exercise 3)

Syntax notes: MERGE, PIVOT, UNPIVOT, GROUPING SETS/CUBE/ROLLUP, and recursive CTEs are written in T-SQL (SQL Server) syntax, since this is the dialect where all of these features share a consistent syntax. Dialect-specific alternatives (PostgreSQL, MySQL) are called out where relevant.

Exercise 1 — Ranking and Window Functions

Goal

Use ROW_NUMBER(), RANK(), DENSE_RANK(), OVER(), and PARTITION BY to find the top 3 most expensive products in each category, and compare how each ranking function handles tied prices.

Step 1 & 3 — ROW_NUMBER() with PARTITION BY / ORDER BY

ROW_NUMBER() assigns a unique, sequential integer within each partition, regardless of ties. Here the data is partitioned by Category and ordered by Price DESC, then filtered to the top 3 rows per category.

```
WITH RankedProducts AS (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        ROW_NUMBER() OVER (
            PARTITION BY Category
            ORDER BY Price DESC
        ) AS RowNum
    FROM Products
)
SELECT ProductID, ProductName, Category, Price, RowNum
FROM RankedProducts
WHERE RowNum <= 3
ORDER BY Category, RowNum;
```

Step 2 — Comparing RANK() and DENSE_RANK()

RANK() and DENSE_RANK() assign the same rank to tied Price values, but differ in how the next rank is numbered: RANK() skips ranks equal to the number of ties, DENSE_RANK() does not.

```
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS RowNum,
    RANK()       OVER (PARTITION BY Category ORDER BY Price DESC) AS PriceRank,
    DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS PriceDenseRank
FROM Products
ORDER BY Category, Price DESC;
```

Top 3 per category using DENSE_RANK() (ties preserved)

```
WITH RankedProducts AS (
    SELECT ProductID, ProductName, Category, Price,
           DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS PriceDenseRank
    FROM Products
)
SELECT * FROM RankedProducts
WHERE PriceDenseRank <= 3
```

```
ORDER BY Category, PriceDenseRank;
```

How ties are handled — worked example

In the Electronics category, two products tie for 2nd place at $499.99:

Product	Price	ROW_NUMBER()	RANK()	DENSE_RANK()
Laptop Pro	$1,299.99	1	1	1
Wireless Headphones	$499.99	2	2	2
Smartwatch	$499.99	3	2	2
Bluetooth Speaker	$249.99	4	4	3

ROW_NUMBER() — always unique; breaks ties arbitrarily; never repeats a number.
RANK() — ties share a rank, then the next rank skips ahead (1, 2, 2, 4).
DENSE_RANK() — ties share a rank, with no gap afterward (1, 2, 2, 3).

Key Takeaways

• PARTITION BY resets the ranking counter for each Category.
• ORDER BY Price DESC inside OVER() controls ranking order (highest price = rank 1).
• ROW_NUMBER() guarantees exactly 3 rows per category; RANK()/DENSE_RANK() may return more than 3 rows if there's a tie at the 3rd position.

Exercise 2 — GROUPING SETS, CUBE, and ROLLUP

Goal

Analyze total quantity sold by Region and Category using GROUPING SETS, ROLLUP, and CUBE.

Step 1 — Base join

All three techniques build on the same underlying join across Orders, OrderDetails, Customers, and Products, aggregating Quantity:

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN Products p ON p.ProductID = od.ProductID
GROUP BY c.Region, p.Category;
```

Step 2 — GROUPING SETS

GROUPING SETS lets you request several independent grouping levels in a single query: totals by Region alone, totals by Category alone, and totals by the Region/Category combination.

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN Products p ON p.ProductID = od.ProductID
GROUP BY GROUPING SETS (
    (c.Region),
    (p.Category),
    (c.Region, p.Category)
)
ORDER BY c.Region, p.Category;
```

Step 3 — ROLLUP

ROLLUP produces a hierarchical set of subtotals — Region+Category detail rows, then a subtotal per Region, then a single grand total row (both columns NULL) — following the column order given.

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN Products p ON p.ProductID = od.ProductID
```

```
GROUP BY ROLLUP (c.Region, p.Category)
ORDER BY c.Region, p.Category;
```

Step 4 — CUBE

CUBE produces every possible combination of the grouping columns: Region+Category detail, Region-only subtotals, Category-only subtotals, and the grand total.

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN Products p ON p.ProductID = od.ProductID
GROUP BY CUBE (c.Region, p.Category)
ORDER BY c.Region, p.Category;
```

Distinguishing subtotal rows with GROUPING()

Region or Category shows as NULL both for an actual NULL value and for a subtotal row. GROUPING() disambiguates: it returns 1 for a generated subtotal row and 0 for a real data value.

```
SELECT
    c.Region,
    p.Category,
    SUM(od.Quantity) AS TotalQuantity,
    GROUPING(c.Region)   AS IsRegionSubtotal,
    GROUPING(p.Category) AS IsCategorySubtotal
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Customers c ON c.CustomerID = o.CustomerID
JOIN Products p ON p.ProductID = od.ProductID
GROUP BY CUBE (c.Region, p.Category)
ORDER BY c.Region, p.Category;
```

Comparison

Technique	Grouping levels produced
GROUPING SETS	Only the exact combinations you list — full control, no extras.
ROLLUP(A, B)	(A,B), (A), () — a hierarchical drill-up in the order given.
CUBE(A, B)	(A,B), (A), (B), () — every possible combination of the columns.

Note: MySQL does not support GROUPING SETS or CUBE directly (ROLLUP is supported via WITH ROLLUP appended to GROUP BY). PostgreSQL supports all three natively, matching the syntax above.

Exercise 3 — CTEs and MERGE

Goal

Use a recursive CTE to generate a calendar table, and a MERGE statement to synchronize product prices from a staging table.

Part A — Recursive CTE: calendar table for January 2025

A recursive CTE has an anchor member (the starting date) and a recursive member that adds one day at a time, referencing itself until the terminating condition is met.

```
WITH DateSequence AS (
    -- Anchor member: starting date
    SELECT CAST('2025-01-01' AS DATE) AS CalendarDate

    UNION ALL

    -- Recursive member: add one day until we pass the end date
    SELECT DATEADD(DAY, 1, CalendarDate)
    FROM DateSequence
    WHERE CalendarDate < '2025-01-31'
)
SELECT CalendarDate
FROM DateSequence
OPTION (MAXRECURSION 100);   -- SQL Server safeguard against infinite recursion
```

> PostgreSQL equivalent: replace DATEADD(DAY, 1, CalendarDate) with CalendarDate + INTERVAL '1 day', and omit the OPTION (MAXRECURSION ...) clause.

Part B — Staging table and MERGE

Step 2: create a StagingProducts table holding updated or new price data:

```
CREATE TABLE StagingProducts (
    ProductID   INT,
    ProductName VARCHAR(100),
    Category    VARCHAR(50),
    Price       DECIMAL(10,2)
);

-- Example staging rows: ProductID 101 gets a price update,
-- ProductID 999 is a brand-new product not yet in Products
INSERT INTO StagingProducts (ProductID, ProductName, Category, Price)
VALUES
    (101, 'Wireless Headphones', 'Electronics', 449.99),
    (999, 'Smart Home Hub',      'Electronics', 129.99);
```

Step 3: MERGE compares Products (target) against StagingProducts (source) on ProductID — updating the price where the product already exists, and inserting it where it doesn't:

```
MERGE INTO Products AS target
USING StagingProducts AS source
    ON target.ProductID = source.ProductID
```

```
WHEN MATCHED THEN
    UPDATE SET
        target.Price       = source.Price,
        target.ProductName = source.ProductName,
        target.Category    = source.Category
WHEN NOT MATCHED BY TARGET THEN
    INSERT (ProductID, ProductName, Category, Price)
    VALUES (source.ProductID, source.ProductName, source.Category, source.Price);
```

MySQL has no MERGE statement; the equivalent is INSERT ... ON DUPLICATE KEY UPDATE. PostgreSQL supports MERGE natively from version 15 onward, and also offers INSERT ... ON CONFLICT DO UPDATE as an alternative.

Exercise 4 — PIVOT and UNPIVOT

Goal

Show monthly sales quantity per product in a pivoted (one column per month) format, then reverse the transformation with UNPIVOT.

Step 1 — Aggregate sales by Product and Month

```
SELECT
    p.ProductName,
    MONTH(o.OrderDate) AS OrderMonth,
    SUM(od.Quantity)   AS TotalQuantity
FROM Orders o
JOIN OrderDetails od ON od.OrderID = o.OrderID
JOIN Products p ON p.ProductID = od.ProductID
GROUP BY p.ProductName, MONTH(o.OrderDate);
```

Step 2 — PIVOT: rows into columns

PIVOT turns the distinct OrderMonth values into their own columns, aggregating Quantity into each. The source query is wrapped as a derived table before applying PIVOT.

```
SELECT
    ProductName,
    [1] AS Jan, [2] AS Feb, [3] AS Mar, [4] AS Apr,
    [5] AS May, [6] AS Jun, [7] AS Jul, [8] AS Aug,
    [9] AS Sep, [10] AS Oct, [11] AS Nov, [12] AS Dec
FROM (
    SELECT
        p.ProductName,
        MONTH(o.OrderDate) AS OrderMonth,
        od.Quantity
    FROM Orders o
    JOIN OrderDetails od ON od.OrderID = o.OrderID
    JOIN Products p ON p.ProductID = od.ProductID
) AS SourceData
PIVOT (
    SUM(Quantity)
    FOR OrderMonth IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])
) AS PivotTable
ORDER BY ProductName;
```

Step 3 — UNPIVOT: columns back into rows

UNPIVOT reverses the transformation, turning the twelve month columns back into MonthName / Quantity row pairs.

```
SELECT
    ProductName,
    MonthName,
    Quantity
FROM (
    SELECT
```

```
        ProductName,
        [1] AS Jan, [2] AS Feb, [3] AS Mar, [4] AS Apr,
        [5] AS May, [6] AS Jun, [7] AS Jul, [8] AS Aug,
        [9] AS Sep, [10] AS Oct, [11] AS Nov, [12] AS Dec
    FROM (
        SELECT p.ProductName, MONTH(o.OrderDate) AS OrderMonth, od.Quantity
        FROM Orders o
        JOIN OrderDetails od ON od.OrderID = o.OrderID
        JOIN Products p ON p.ProductID = od.ProductID
    ) AS SourceData
    PIVOT (
        SUM(Quantity)
        FOR OrderMonth IN ([1],[2],[3],[4],[5],[6],[7],[8],[9],[10],[11],[12])
    ) AS PivotTable
) AS PivotedResult
UNPIVOT (
    Quantity FOR MonthName IN (Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec)
) AS UnpivotedResult
ORDER BY ProductName, MonthName;
```

PIVOT and UNPIVOT are T-SQL-specific. PostgreSQL achieves the same result with the crosstab() function (tablefunc extension) for pivoting, and a UNION ALL of the individual month columns for unpivoting. MySQL requires conditional aggregation (SUM(CASE WHEN OrderMonth = 1 THEN Quantity END) AS Jan, ...) in place of PIVOT.

Exercise 5 — Using a CTE to Simplify a Query

Goal

Use a Common Table Expression to find all customers who have placed more than 3 orders in total.

Step 1 & 2 — CTE for order counts, then filter

The CTE isolates the aggregation (counting orders per customer) so the outer query stays a simple, readable join and filter — this is the pattern given in the exercise itself:

```
WITH CustomerOrderCounts AS (
    SELECT
        o.CustomerID,
        COUNT(o.OrderID) AS OrderCount
    FROM Orders o
    GROUP BY o.CustomerID
)
SELECT
    c.CustomerID,
    c.Name,
    coc.OrderCount
FROM CustomerOrderCounts coc
JOIN Customers c ON c.CustomerID = coc.CustomerID
WHERE coc.OrderCount > 3
ORDER BY coc.OrderCount DESC;
```

Why use a CTE here?

• Readability — the aggregation logic (counting orders) is named and separated from the filtering/join logic, so each part of the query answers one question at a time.
• Reusability — CustomerOrderCounts could be referenced multiple times in the same statement without repeating the GROUP BY subquery.
• Without a CTE, this would require either a HAVING clause on the same query (fine here, but not for a two-step aggregate join) or a nested subquery in the FROM clause, which reads less naturally.

Equivalent without a CTE (for comparison)

```
SELECT
    c.CustomerID,
    c.Name,
    COUNT(o.OrderID) AS OrderCount
FROM Customers c
JOIN Orders o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING COUNT(o.OrderID) > 3
ORDER BY OrderCount DESC;
```

Both queries return the same result. The HAVING version is more compact for this single aggregation; the CTE version scales better once more logic needs to build on CustomerOrderCounts.